

# INFORME DE ESTADO DE PRUEBAS

## Swagger PetStore

*Reporte de resultados y evidencias*

### Resumen

Se ejecutó la colección completa de pruebas sobre Swagger PetStore. Durante la validación se detectó un bug crítico relacionado con el proceso de autenticación y se verificó una diferencia de comportamiento entre Postman y Newman.

### Test result de la collection en postman

Proyecto PetStore - Run results

🔄

📺

+ New Run

📄

Share

🔗

...

🕒

Ran today at 07:11:09 PM · [View all runs](#)

| Source | Environment       | Iterations | Duration | All tests | Errors | Avg. Resp. Time |
|--------|-------------------|------------|----------|-----------|--------|-----------------|
| Runner | variables de test | 1          | 3s 787ms | 25        | 0      | 246 ms          |

RUN SUMMARY

List

Grid

1

> POST CreateNewUser

3 | 0

✓

✗

✓ GET CheckExistUser

2 | 1

✗

✗

PASS CHK1- Validate Status code is 200

PASS CHK2: Validate Response time is less than 950ms

FAIL CHK3-Validate Body is correct

> GET CheckUpdateUser

3 | 0

✓

✗

> GET CheckUnexistUser

3 | 0

✓

✗

> PUT UpdateExistUser

3 | 0

✓

✗

> DELETE DeleteExistUser

3 | 0

✓

✗

> DELETE DeleteUnexistUser

2 | 0

✓

✗

> GET LoginSuccessfulExistUser

3 | 0

✓

✗

✓ GET LoginUnsuccessfulUnexistUser

1 | 1

✗

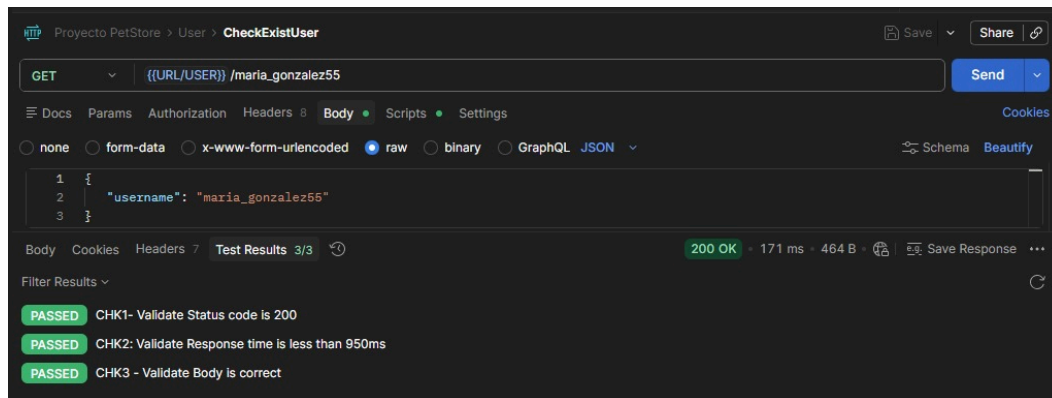
✗

FAIL CHK1- Validate Status code is 400

PASS CHK2: Validate Response time is less than 200ms

## Corrección de prueba

El test 'CheckExistUser' presentaba un error de tipo en la validación. Luego de corregirlo, la prueba finalizó exitosamente.

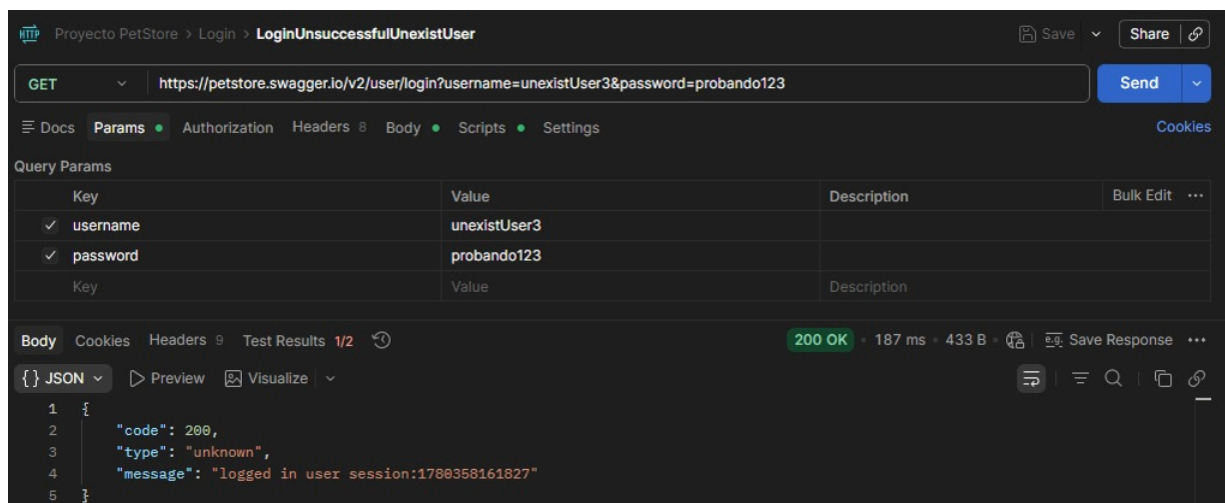


## DETECCIÓN DE BUG: Login de usuario inexistente

El caso 'LoginUnsuccessfulUnexistUser' reveló un comportamiento inesperado: el servidor permite autenticarse con un usuario que no existe en el sistema.

Impacto:

- Riesgo de seguridad.
- Validación insuficiente de autenticación.
- Posibilidad de accesos no previstos.



La consulta realizada sobre Swagger confirmó que el usuario utilizado en la prueba no se encontraba registrado, validando el hallazgo reportado.

GET

/user/{username} Get user by user name

^

Parameters

Cancel

| Name                                                                         | Description                                               |
|------------------------------------------------------------------------------|-----------------------------------------------------------|
| <b>username</b> * <small>required</small><br>string<br><small>(path)</small> | The name that needs to be fetched. Use user1 for testing. |

unexistUser3

Execute

Clear

Responses

Response content type application/json

Curl

```
curl -X 'GET' \
  'https://petstore.swagger.io/v2/user/unexistUser3' \
  -H 'accept: application/json'
```

Request URL

https://petstore.swagger.io/v2/user/unexistUser3

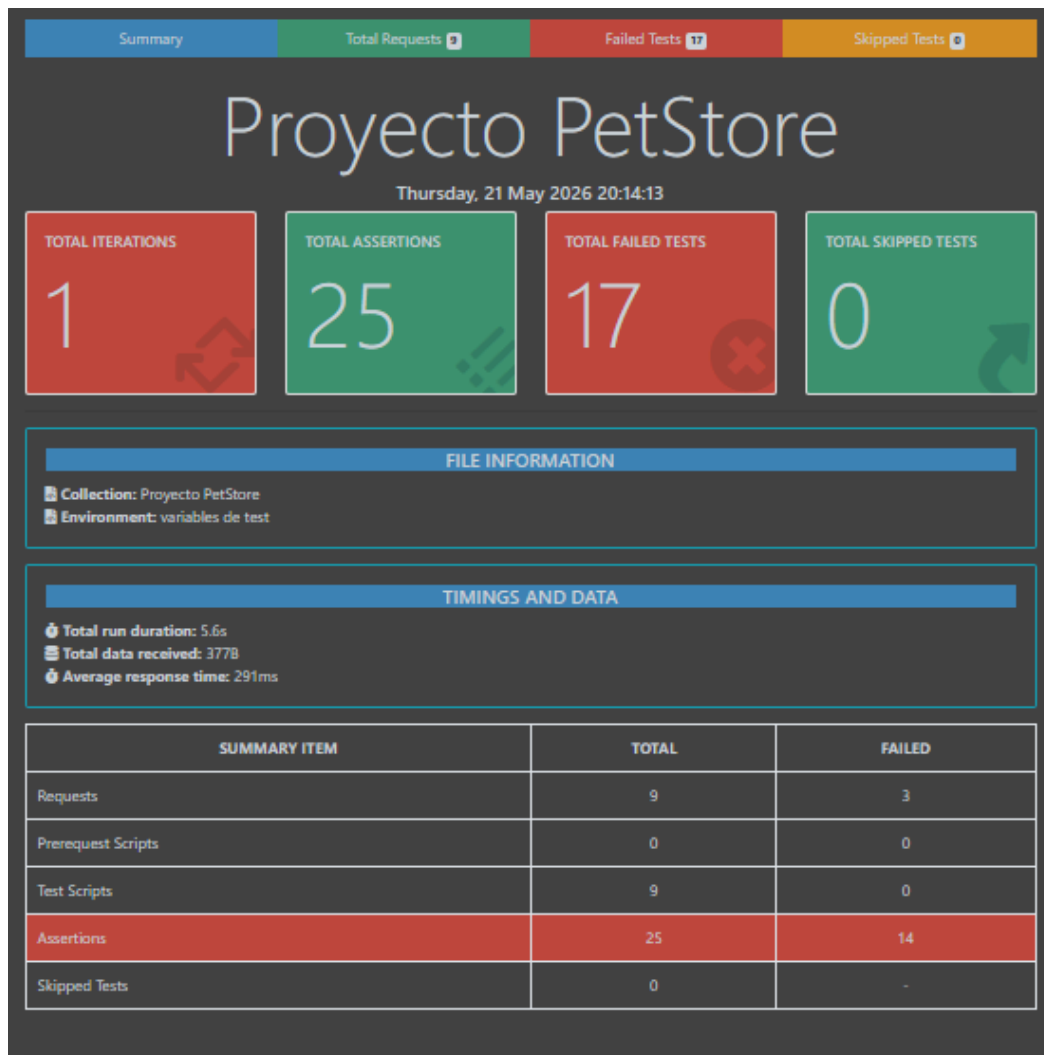
Server response

| Code | Details                                                                                                                                                                                  |
|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 404  | <div>Error: response status is 404</div> <div>Response body</div> <div><pre>{   "code": 1,   "type": "error",   "message": "User not found" }</pre></div> <div><div>Download</div></div> |

## REPORTE DE EJECUCIÓN CON NEWMAN

Luego de investigar estos resultados, se definio que en **Postman**, las pruebas se ven correctas porque la aplicación maneja automáticamente las variables y muestra las respuestas tal como se esperan. Esto hace que los tests pasen sin problemas.

En **Newman**, al ejecutar desde la consola, el reporte refleja de manera más estricta lo que realmente devuelve el servidor. Si la respuesta no coincide exactamente con lo que el test espera, aparece como fallo en el informe, aunque en Postman se muestre bien.



The screenshot displays the 'Summary' tab of a Postman test suite. At the top, four colored bars indicate the status: 'Summary' (blue), 'Total Requests' (green, 9), 'Failed Tests' (red, 17), and 'Skipped Tests' (orange, 0). Below this, a section titled '1 ITERATION AVAILABLE TO VIEW' includes buttons for 'Show Failed Iterations', 'Expand Folders', and 'Expand Requests'. A red box with the number '1' indicates the selected iteration. The text 'ITERATION 1 SELECTED' is shown below. The test suite is organized into two main folders, each with a dropdown arrow and a red status icon: 'USER - 7 REQUESTS IN THE FOLDER' and 'LOGIN - 2 REQUESTS IN THE FOLDER'. Under the 'USER' folder, there are seven test items: 'Iteration: 1 - CreateNewUser', 'Iteration: 1 - CheckExistUser', 'Iteration: 1 - CheckUpdateUser', 'Iteration: 1 - CheckUnexistUser', 'Iteration: 1 - UpdateExistUser' (highlighted in green), 'Iteration: 1 - DeleteExistUser', and 'Iteration: 1 - DeleteUnexistUser' (highlighted in green). Under the 'LOGIN' folder, there are two test items: 'Iteration: 1 - LoginSuccessfulExistUser' and 'Iteration: 1 - LoginUnsuccessfulUnexistUser'. All test items have a red status icon and a left-pointing arrow.

## Recomendación

Se precisa solucionar bug de severidad crítica para fortalecer la seguridad de la web y que no se permita el login de usuarios que no están registrados. Corregir el flujo de autenticación, y solucionar ambigüedad entre postman y newman.